

Churn

November 5, 2025

```
[9]: import pandas as pd
import numpy as np
```

```
[10]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
[11]: df = pd.read_csv("Churn_Modelling.csv")
```

```
[12]: print(df.head())
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

```
[13]: print(df.shape)
```

(10000, 14)

```
[14]: df.head()
```

```
[14]:   RowNumber  CustomerId  Surname  CreditScore  Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female  42
1          2    15647311    Hill          608    Spain  Female  41
2          3    15619304    Onio          502    France  Female  42
3          4    15701354    Boni          699    France  Female  39
4          5    15737888  Mitchell          850    Spain  Female  43

      Tenure  Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0          2      0.00              1          1              1
1          1  83807.86              1          0              1
2          8 159660.80              3          1              0
3          1      0.00              2          0              0
4          2 125510.82              1          1              1

      EstimatedSalary  Exited
0          101348.88        1
1          112542.58        0
2          113931.57        1
3           93826.63        0
4           79084.10        0
```

```
[18]: # pip install tensorflow ignore if already installed
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
[19]: # 2. Select features and target
# Remove columns that are clearly not useful for prediction
X = df.drop(columns=['RowNumber', 'CustomerId', 'Surname', 'Exited'])
X
```

```
[19]:   CreditScore  Geography  Gender  Age  Tenure  Balance  NumOfProducts  \
0          619    France  Female  42      2      0.00              1
1          608    Spain  Female  41      1  83807.86              1
2          502    France  Female  42      8 159660.80              3
3          699    France  Female  39      1      0.00              2
4          850    Spain  Female  43      2 125510.82              1
...         ...      ...    ...    ...    ...      ...
9995         771    France    Male  39      5      0.00              2
9996         516    France    Male  35     10  57369.61              1
9997         709    France  Female  36      7      0.00              1
9998         772    Germany    Male  42      3  75075.31              2
9999         792    France  Female  28      4 130142.79              1

      HasCrCard  IsActiveMember  EstimatedSalary
0              1              1          101348.88
```

1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10
...	...	...	...
9995	1	0	96270.64
9996	1	1	101699.77
9997	0	1	42085.58
9998	1	0	92888.52
9999	1	0	38190.78

[10000 rows x 10 columns]

```
[20]: y = df['Exited']
      y
```

```
[20]: 0      1
      1      0
      2      1
      3      0
      4      0
      ..
      9995    0
      9996    0
      9997    1
      9998    1
      9999    0
      Name: Exited, Length: 10000, dtype: int64
```

```
[21]: # Train-test split (80-20 usually good for neural nets)
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
      )
```

```
[22]: X_train.shape, X_test.shape
```

```
[22]: ((8000, 10), (2000, 10))
```

```
[23]: # One-hot encode categorical features
      X_train = pd.get_dummies(X_train, drop_first=True)
      X_test = pd.get_dummies(X_test, drop_first=True)
```

```
[24]: # Scale numeric values
      scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
      X_test = scaler.transform(X_test)
```

```
[26]: model = Sequential([
        Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
        Dropout(0.2),
        Dense(8, activation='relu'),
        Dropout(0.2),
        Dense(1, activation='sigmoid') # Binary classification
    ])
```

```
[27]: model.compile(optimizer='adam', loss='binary_crossentropy',
    ↪ metrics=['accuracy'])
```

```
[28]: history = model.fit(
        X_train, y_train,
        batch_size=32,
        epochs=25,
        validation_data=(X_test, y_test),
        verbose=0
    )
```

```
[29]: y_pred = (model.predict(X_test) > 0.5).astype(int)
```

63/63 0s 2ms/step

```
[30]: print("Accuracy:", accuracy_score(y_test, y_pred))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 0.8555

Confusion Matrix:

```
[[1561  32]
 [ 257 150]]
```

```
[ ]:
```